

Algorithm 1: Data Parallelism Baseline

Comparing Energy Costs of Parallel DNN Training Techniques

Note on AI Usage: Revised with AI writing and coding support (Claude, Anthropic) per CS 4953 LLM use policy..

Note on Timing Data: Source code for all three framework implementations (PyTorch DDP, TensorFlow MirroredStrategy, and AxoNN) is posted to the course GitHub. Timing data files and performance plots are pending cluster runs currently being scheduled on UTSA ARC and Delta and will be posted within the week.

1. Problem

The environmental cost of training AI models has become a real issue. Data centers that run these workloads are consuming massive amounts of electricity and water by 2025, estimates put AI's carbon footprint at the equivalent of New York City and its water usage at the scale of global bottled water consumption annually. Most of the research focus goes toward making models more accurate or faster, but very little asks: which training method actually uses less power to get there?

This project tries to answer that question in a concrete and measurable way. I'm comparing three parallel DNN training techniques PyTorch Distributed Data Parallel (DDP), TensorFlow MirroredStrategy, and AxoNN and measuring not just training speed but actual energy consumption. Algorithm 1 establishes the data parallelism baseline across all three frameworks, so that Algorithms 2 and 3 (mixed precision/quantization and ZeRO memory optimization) have something to compare against.

2. Chosen Algorithm / Method

Data parallelism is the most widely used approach for distributed DNN training and the obvious starting point for this project. The core idea is straightforward: every GPU gets a complete copy of the model, but each one processes a different chunk of the training data at the same time. At the end of each step, the gradients from all GPUs get averaged and the weights get updated. That synchronization step is what makes it work, but it also introduces communication overhead that grows as you add more GPUs and that overhead costs energy.

Rather than implementing data parallelism just once, Professor Prasad's feedback pushed me to compare how three different frameworks handle the same underlying idea. Each one uses a different communication strategy and design philosophy:

Method 1a: PyTorch Distributed Data Parallel (DDP)

PyTorch DDP is the standard baseline. Each GPU runs its own training process, and gradients are synchronized using the NCCL all-reduce operation at the end of each backward

pass. DDP is smart about overlapping gradient communication with computation, as soon as a layer's gradients are ready, they start being sent while backprop continues on earlier layers. This overlap reduces but doesn't eliminate the communication cost. Implementation uses torchrun to launch multiple processes (Li et al., 2020).

Method 1b: TensorFlow MirroredStrategy

TensorFlow's MirroredStrategy is the TF equivalent of DDP; it mirrors the model across all available GPUs and uses all-reduce to synchronize gradients. The main difference is that TF handles distribution at the framework level rather than the process level, which makes it easier to set up but gives less control over the underlying communication. MirroredStrategy also scales the batch size automatically by the number of GPUs, which means the effective learning rate needs to be tuned accordingly.

Method 1c: AxoNN

AxoNN (Singh & Bhatele, 2022) was developed specifically for extreme-scale training and takes a different approach to communication entirely. Instead of synchronous NCCL all-reduce, AxoNN uses asynchronous MPI-based point-to-point messaging, which lets GPUs keep computing while communication happens in the background. It also offloads data to CPU memory periodically to reduce GPU memory pressure. AxoNN was shown to outperform Megatron-LM and DeepSpeed on multi-billion parameter transformer models. For CIFAR-10/ResNet18 at our scale, it will be interesting to see whether the asynchronous overhead advantage holds or whether the simpler synchronous approach of DDP is more efficient.

3. Underlying Communication Pattern

All three methods use some form of gradient averaging across GPUs, but they do it differently:

- PyTorch DDP: NCCL ring all-reduce gradients flow around a ring topology, each GPU sends to its neighbor until all gradients are summed across the group. Synchronous and blocking before the optimizer step.
- TensorFlow MirroredStrategy: Also NCCL all-reduce under the hood, but managed by the TF runtime. Similar pattern to DDP but with additional TF-side overhead from the graph execution model.
- AxoNN: Asynchronous MPI point-to-point, instead of waiting for a full all-reduce, GPUs send and receive gradients between neighbors without blocking. Computation and communication overlap more aggressively than in DDP.

The all-reduce communication volume per step is approximately proportional to the number of model parameters. For ResNet18, that's roughly 11 million parameters, or about 44MB in FP32. This stays constant per step regardless of GPU count, which means communication becomes a larger fraction of total time as training steps get faster with more GPUs.

4. Data Structures, Datasets, and Hyperparameters

Model: ResNet18, trained from scratch (no pretrained weights), modified for 10-class output to match CIFAR-10.

Dataset: CIFAR-10, 60,000 32x32 color images across 10 categories (50,000 train, 10,000 test). Each GPU receives a non-overlapping shard via DistributedSampler (PyTorch) or automatic sharding (TF/AxoNN).

Hyperparameters:

- Batch size: 128 per GPU
- Learning rate: 0.1 with cosine annealing over 10 epochs
- Optimizer: SGD with momentum 0.9, weight decay $5e-4$
- Epochs: 10 per run
- Precision: FP32 (full precision baseline, mixed precision is Algorithm 2)
- GPU counts tested: 1 GPU (baseline), 2 GPUs (parallel)

5. Read/Write Contention and Synchronization Overheads

In DDP, the main synchronization point is the all-reduce at the end of each backward pass. Every GPU must wait for this to complete before the optimizer updates weights, this is a hard synchronization barrier. DDP mitigates this by bucketing gradients and overlapping communication with computation, but the barrier still exists.

Read/write contention is relatively low in DDP compared to parameter server approaches because there's no central node being written to by all workers simultaneously. All-reduce is symmetric, so no single node is a bottleneck. In AxoNN, the asynchronous model removes the hard barrier entirely but introduces the risk of stale gradients if a GPU finishes a step before receiving all updates, this is the main tradeoff between synchronous and asynchronous approaches.

Memory-wise, each GPU in DDP holds the full model, optimizer states, gradients, and activations, which for ResNet18 is approximately 500MB–1GB depending on batch size and framework overhead. This is well within the memory limits of the cluster GPUs, so memory contention is not expected to be a limiting factor in this experiment.

6. Parallel Time Complexity

For N GPUs, dataset size D , and per-GPU batch size B :

- Computation per step: $O(B \times \text{model_FLOPs})$, constant per GPU regardless of N , since each GPU only sees $1/N$ of the data
- Communication per step: $O(P / \text{bandwidth})$ where $P \approx 44\text{MB}$ for ResNet18, constant regardless of N for ring all-reduce
- Steps per epoch: $O(D / (N \times B))$, decreases linearly with GPU count
- Total time per epoch: $O((D / (N \times B)) \times (t_{\text{compute}} + t_{\text{communicate}}))$

Ideal speedup would be linear in N . In practice, speedup is sublinear because $t_{\text{communicate}}$ stays roughly constant while t_{compute} per step decreases. As N grows, communication becomes a larger fraction of total time and efficiency drops. For ResNet18 at this scale, speedup from 1 to 2 GPUs should be close to 1.6–1.8x based on prior literature.

7. Timing and Experiment Details

Each configuration will be run at least 3 times to average out variance from cluster scheduling and GPU memory state. The following parameters will be varied:

- Number of GPUs: 1 vs. 2
- Framework: PyTorch DDP vs. TensorFlow MirroredStrategy vs. AxoNN
- Batch size: 64 and 128 (to observe memory and throughput tradeoffs)

Metrics collected per epoch per run:

- Epoch wall-clock time (seconds), measured with `Python time.time()`
- GPU memory allocated and reserved (MB), via `torch.cuda.memory_allocated()` for PyTorch, `tf.config.experimental.get_memory_info()` for TF
- Power draw (Watts), via `nvidia-smi` polling every 5 seconds, logged to CSV
- Training loss and test accuracy per epoch

Energy per epoch will be estimated as the integral of power over time: $\text{Energy (Joules)} = \text{average power draw (W)} \times \text{epoch duration (s)}$. This methodology follows the approach described in the IEEE MICRO 2025 paper on characterizing distributed training efficiency (cited in bibliography).

8. Performance Results

Note: Timing data and plots are pending cluster runs scheduled on UTSA ARC and Delta. Results will be posted to the course GitHub within the week. The following metrics will be reported once available:

- (a) Execution/training time per epoch vs. number of GPUs for each framework
- (b) Speedup: time with 1 GPU / time with N GPUs, plotted against ideal linear speedup
- (c) Test accuracy across epochs for each framework
- (d) Estimated energy consumption (Joules) per epoch per framework
- (e) Power draw over time (W) from `nvidia-smi` logs

Plots will be generated in Python using `matplotlib` and posted alongside the CSV timing files on GitHub.

9. Conclusions (Preliminary)

Data parallelism with PyTorch DDP is the most straightforward of the three frameworks to implement and the obvious baseline. Based on what I've learned in class and from the papers, I expect doubling GPU count will reduce training time but not by a full 2x, the all-reduce communication overhead eats into the speedup, and both GPUs are drawing power simultaneously, so the energy per epoch might not decrease proportionally either.

The more interesting comparison will be between the three frameworks. TensorFlow MirroredStrategy and PyTorch DDP use the same underlying all-reduce approach, so differences there will come down to framework overhead and implementation details. AxoNN's asynchronous communication model should show its advantage more clearly at larger scales, but at the ResNet18/CIFAR-10 level the overhead of MPI setup might offset the gains.

What I'm most curious about is whether there's a framework that is both faster AND uses less energy, or if the faster one always uses more power to achieve the speedup. If energy and speed don't track each other, that has real implications for how we should be choosing training infrastructure, which is exactly the question this project is trying to answer going into Algorithms 2 and 3.

References

- Li, S., et al. "PyTorch Distributed: Experiences on Accelerating Data Parallel Training." VLDB, 2020. <https://arxiv.org/abs/2006.15704>
- Singh, S., Bhatele, A. "AxoNN: An Asynchronous, Message-Driven Parallel Framework for Extreme-Scale Deep Learning." IPDPS, 2022. <https://arxiv.org/abs/2110.13005>
- Sergeev, A., Del Balso, M. "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow." arXiv, 2018. <https://arxiv.org/abs/1802.05799>
- "Characterizing the Efficiency of Distributed Training: A Power, Performance, and Thermal Perspective." IEEE MICRO, 2025. <https://dl.acm.org/doi/10.1145/3725843.3756111>
- Mahmud, H. "Converting Autoencoder Based Energy-Efficient and Secure DNN Inference on Edge Devices." PhD Dissertation, UTSA, 2025.
- Rajbhandari, S., et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." SC'20, 2020. <https://arxiv.org/abs/1910.02054>